

第 5 次课教师讲义：分支、循环与汇编程序设计基础

技术编号：week05 | 建议检查点：week05-branch-loop
把“会算、会访存”升级为“会设计有结构的汇编程序”。

0. 一句话目标

掌握分支/循环指令与结构化程序设计方法，能独立写出计数循环与字符串遍历，并读透 `clear_bss` 控制流。

1. 课程定位 (Why)

到第 4 次课为止，学生可以写出直线指令序列。真实程序需要：

判断 → 分支
重复 → 循环（条件 + 回跳）

第 2 次课的 `clear_bss` 已经是标准 `while` 结构；本次课把它一般化为程序设计能力。

2. 教学目标

- 1. 掌握 `b / beq / bne / beqz / bnez`。
- 2. 会把 `if / while / for` 翻译成汇编排版。
- 3. 会写字符串扫描骨架（为 `strlen/uart_puts` 铺路）。
- 4. 养成：先画流程图，再写指令，再走寄存器模拟。
- 5. 能指出死循环、漏更新游标、条件写反等典型错误。

3. 课前准备

- 准备 `if/while` 流程图模板。
- 打开 `boot/start.S` 的 `clear_bss`、`lib/string.S` 的 `strlen`（可先不讲参数约定细节）。

4. 课程知识

4.1 控制流指令家族

指令	含义	C 直觉
<code>b L</code>	无条件转到 L	<code>goto L</code>
<code>beq rs, rt, L</code>	相等则转	<code>if (rs==rt) goto L</code>
<code>bne rs, rt, L</code>	不等则转	<code>if (rs!=rt) goto L</code>
<code>beqz rs, L</code>	为 0 则转	<code>if (rs==0) goto L</code>
<code>bnez rs, L</code>	非 0 则转	<code>if (rs!=0) goto L</code>

4.2 局部标号 1b / 2f

- 1b：向后找数字标号 1 (back)
 - 2f：向前找数字标号 2 (forward)
- 便于写短循环，避免想名字。

4.3 结构化模板

```
# $a0 = n, 执行 n 次 body
loop:
    beqz    $a0, done
    # ... body ...
    addi.d  $a0, $a0, -1
    b       loop
done:
```

汇编精讲: **while** 计数

1. 先判空: $n==0$ 直接结束。
2. 体与步进分离。
3. **b loop** 回到条件。

```
    beqz    $a0, else_part
    # then
    b       endif
else_part:
    # else
endif:
```

汇编精讲: **if-else** 强调: then 结束要跳过 else, 否则两路都执行。

```
# $a0 指向 C 字符串
1:
    ld.bu   $t0, $a0, 0
    beqz    $t0, 2f
    # 可在此处理 $t0 字符
    addi.d  $a0, $a0, 1
    b       1b
2:
```

汇编精讲: 字符串遍历

1. 读字节 → 判 '\0' → 处理 → 指针 ++ → 回跳。
2. 这是 strlen、uart_puts 的共同骨架。

```
1:
    beq     $t0, $t1, 2f
    st.b    $zero, $t0, 0
    addi.d  $t0, $t0, 1
    b       1b
2:
```

汇编精讲: 回看 **clear_bss** 对照 while: 条件是“游标未到终点”, 体是“写 0”, 步进是“地址 +1”。

4.4 汇编程序设计方法（本课核心方法论）

写程序前强制五问：

1. 输入在哪些寄存器/内存？
2. 输出写到哪里？
3. 哪些寄存器可作临时？会不会破坏仍需的值？
4. 循环不变量是什么？（例如：`$t0` 始终指向下一个待清零字节）
5. 是否必然终止？游标/计数是否更新？

推荐流程：

自然语言 → 流程图 → 伪代码 → 汇编 → 寄存器模拟走两遍 → 再上机

5. 课堂 Demo

1. 师生共译：`sum = 1..n`。
2. 故意写错：忘记 `addi`，让学生找死循环。
3. 并排展示 `clear_bss` 与 `strlen` 的结构同构。

6. 实验实践

Task A

实现 `sum=1..n` (`n` 在 `$a0`，和在 `$a0` 返回)。

Task B

实现 `strlen` 骨架（不含调用约定细节也可，返回长度在 `$a0`）。

Task C

为 `clear_bss` 画流程图，并标注每条指令属于“条件/体/步进/退出”。

Task D (选做)

输出固定字符串 `N` 次（与后续 `UART` 结合）。

7. AI 共学

允许生成流程图与对照检查；禁止整段不可读粘贴且自己讲不清。

8. 思考与拓展

1. 空字符串时循环体执行几次？
2. `b 1b` 与 `bl 1b` 混用会造成什么？
3. 编译器把 `for` 译成汇编时，最常见的落点指令是什么？

9. 板书

条件分支：`beq/beqz/bnez`

循环 = 判 + 体 + 步进 + 回跳

字符串：`ld.bu` → `beqz` → ... → `addi` → `b`

方法：五问 + 流程图优先